

INTERNSHIP COMPLETION REPORT

SupportPilot — AI-Powered Customer Support Platform with Ticket Resolution Agent

Team Details

Field	Details
Team Number	Team 1
Start Date	15 July 2026
Name(s)	Syed Humera Tasleem
Internship Duration	8 Weeks
Project Code	SP-ATR-001

1. Project Title

AI-Powered Customer Support Platform with Ticket Resolution Agent

2. Project Objective

The objective of this internship project was to design and build SupportPilot, an AI-powered IT support ticket resolution platform capable of automatically classifying incoming support tickets, retrieving relevant knowledge to ground a proposed resolution, coordinating a multi-agent workflow that connects the resolution to real enterprise systems (Jira and email), and finally converting that AI output into a verified, human-approved customer resolution with a controlled closure process.

The project was structured into four sequential milestones, each building directly on the outputs of the one before it:

- Milestone 1 — Ticket intake, preprocessing, and classification (category, sub-category, severity, priority, and SLA).
- Milestone 2 — Knowledge base ingestion and Retrieval-Augmented Generation (RAG) to produce cited, grounded troubleshooting resolutions.
- Milestone 3 — A multi-agent orchestration layer that consumes the M1/M2 outputs and drives Jira ticket management and automated email communication.

- Milestone 4 — AI resolution validation, agent collaboration, escalation, customer confirmation, and controlled ticket closure — the human-in-the-loop layer that converts an AI-generated answer into a verified, auditable support outcome.

This mirrors how real enterprise support desks operate: a ticket must first be understood and triaged correctly, then matched against organisational knowledge, coordinated through an automated workflow, and only then confirmed and closed with a human accountable for the outcome. The relevance to the organisation is direct: reducing manual triage effort, cutting resolution time on high-volume, well-documented issues, ensuring high-risk or low-confidence cases always reach a human, and giving the business a measurable, auditable definition of what counts as a successfully completed ticket.

3. Project description in detail

3.1 Overall Approach

The project followed a milestone-based build methodology, with each of the four milestones having its own frozen scope, exit criteria, and Definition of Done, signed off before the next milestone began. A recurring design principle across all milestones was to keep computationally expensive components (LLM calls, full human review) reserved for genuinely uncertain or high-risk cases, while cheap deterministic logic and confidence-based routing handled the majority of routine volume. This 'fast path vs. slow path' philosophy shows up in ticket classification (M1), retrieval and generation (M2), workflow orchestration (M3), and finally in confidence-based auto-response versus agent-review versus escalation routing (M4).

3.2 Milestone 1 — Ticket Processing & Classification (Weeks 1–2)

Milestone 1 established the intake and triage foundation of the platform. Before any code was written, a category taxonomy (nine top-level categories such as NETWORK, VPN, ACCESS, SOFTWARE, HARDWARE, PRINTER, EMAIL, SECURITY, APPLICATION, plus an UNCLASSIFIED catch-all) and a severity/priority model were frozen in a working session with the support desk lead, since changing the taxonomy after data collection would invalidate all labelled training examples.

Key components delivered:

- A fully validated multi-section ticket submission form (Issue, Impact, Context, Requester) with Zod-based validation, live classification preview, and duplicate-ticket detection.
- Preprocessing utilities — `strip_email_artifacts` (removing quoted replies and signatures) and `mask_pii` (masking emails, phone numbers, IPs, employee IDs, and card-like numbers before storage or any external model call).
- A hybrid classification dispatcher: a fast supervised path (LightGBM/logistic regression over embeddings and structured features) handles the majority of tickets in under 50 ms, escalating only low-confidence, rare-class, or security-flagged tickets to an LLM-based structured-output classifier.
- A deterministic severity-override layer that can only raise (never lower) a predicted severity — for example, VIP requesters, organisation-wide impact, or outage-related phrases automatically floor the severity at HIGH or CRITICAL.
- A pure matrix lookup mapping severity $\times$ impact scope to priority (P1–P4), and an SLA engine that computes first-response and resolution due-dates against per-site business calendars.
- An agent-facing ticket queue, list, and detail view, plus a manual classification-override feature that captures every agent correction as a labelled training example for future model retraining.

Exit criteria: $\geq 90\%$ ticket classification accuracy and $\geq 85\%$ severity prediction accuracy on a frozen, time-split 500-ticket hold-out set, evaluated with full per-class confusion matrices rather than a single headline accuracy figure, since class-level confusions (e.g., NETWORK vs. VPN) matter more operationally than overall accuracy.

3.3 Milestone 2 — Knowledge Retrieval & Resolution Generation (Weeks 3–4)

Milestone 2 transformed a correctly classified ticket into a cited, step-by-step suggested resolution using Retrieval-Augmented Generation. The milestone began with a Knowledge Base Coverage Audit — cross-referencing the top ticket categories by volume (from Milestone 1 data) against existing documentation to flag GOOD, THIN, or MISSING coverage, since retrieval quality is fundamentally capped by the quality and completeness of the underlying knowledge corpus.

Key components delivered:

- An ingestion pipeline (Load → Normalise → Chunk → Embed → Verify) supporting Markdown, PDF, DOCX, HTML, and Confluence exports, with heading-aware chunking (splitting on H1–H3 boundaries, 500–800 token budgets, 15% overlap, and heading-path prefixes) so that chunks remain semantically self-contained.
- A hybrid retrieval pipeline: messy ticket text is first distilled into 2–3 focused search queries via a lightweight LLM call, then searched using both MongoDB Atlas Vector Search (semantic) and Atlas Search (keyword/BM25) in parallel, fused with Reciprocal Rank Fusion ($k = 60$), filtered, and reranked with a cross-encoder down to the top 5 chunks.
- A grounded generation step that produces JSON-structured resolution steps, where every step must cite a specific knowledge chunk; steps without a resolvable citation are dropped and logged as hallucination events, and destructive actions (delete, reset, credential change) are independently flagged for mandatory human approval.
- A deliberate refusal path: when retrieved context is insufficient, the system returns no steps and instead records a Knowledge Base Gap entry for the Knowledge Manager, rather than generating a plausible but ungrounded answer.
- An agent console Resolution Panel (rendered strictly as a 'Draft — not sent' state, since nothing is auto-sent in this milestone) with citation cards linking each step back to its source article.

Exit criteria: $\text{Recall}@5 \geq 0.85$ on a curated 100 ticket→article pair set, and 100% of generated resolutions carrying at least one valid citation, backed by a four-part evaluation harness (retrieval, generation, refusal, and adversarial/safety golden sets) wired into CI.

3.4 Milestone 3 — Multi-Agent Workflows & Integrations (Weeks 5–6)

Milestone 3 did not rebuild ticket classification or knowledge retrieval; it consumed the outputs of Milestones 1 and 2 and coordinated them inside a multi-agent workflow that connects the platform to enterprise systems. The explicit implementation rule for this milestone was: do not duplicate Milestone 2 — the Knowledge Retrieval Agent must call the existing M2 RAG layer rather than building a second retrieval system.

Key components delivered:

- A Multi-Agent Orchestrator that sequences specialised agents — Diagnosis, Knowledge Retrieval, Resolution Generation, and Escalation — and tracks workflow state end to end.
- A Diagnosis Agent that reasons over the ticket and Milestone 1 classification context to identify likely causes and flag missing information.

- A Validation/decision step that checks whether the generated resolution is sufficiently grounded and confident; well-grounded resolutions proceed to automated response, while low-confidence or unresolved cases are routed to a dedicated Escalation Agent for human hand-off.
- A Jira integration layer that creates and updates enterprise issues, keeps status synchronised bidirectionally (In Progress, Escalated, Resolved), and preserves a last-synchronised timestamp.
- An email automation layer covering ticket-received, resolution, escalation, and resolved notifications, each logged with delivery status.
- New auditable data structures — Agent_Workflows, Agent_Executions, Jira_Tickets, and Email_Logs — so that every agent decision, Jira action, and email send is traceable end to end.
- Frontend additions: an AI Agent screen (workflow status, agent timeline, confidence), an Integrations screen (Jira/Email connection status), and enriched Ticket Detail view combining M1 classification, M2 retrieved knowledge, and M3 workflow/Jira/email history in one place.

Completion criteria: multi-agent workflows execute successfully for complex issues, Jira integration functions correctly for ticket management, and email automation reliably delivers support communications — validated with an end-to-end scenario (e.g., a VPN connectivity ticket flowing from classification through diagnosis, retrieval, resolution, Jira creation, and email notification).

3.5 Milestone 4 — AI Resolution Validation, Agent Collaboration & Ticket Closure Workflow (Weeks 7–8)

Milestone 4 is the human-in-the-loop resolution and closure layer that completes the platform. Where M1–M3 understand the ticket, retrieve knowledge, and generate a coordinated agent workflow, M4 ensures that AI output is safely converted into a real, verified customer support action — nothing is treated as 'done' until a resolution has been validated, delivered, confirmed by the customer, and formally closed with full audit history.

Key components delivered:

- A confidence + risk + policy check that classifies every AI-generated suggestion into one of three paths: high-confidence/low-risk cases eligible for an approved automated response, medium-confidence cases requiring mandatory agent validation before sending, and low-confidence or complex cases routed straight to escalation — with thresholds (e.g., 90–100% auto-eligible, 70–89% agent review, below 70% manual investigation) configurable by an Admin rather than hard-coded.
- A Support Agent Validation workflow: an AI Review Queue surfaces medium-confidence or sensitive tickets, and each agent works through a structured checklist — issue understanding, classification/priority accuracy, AI-answer accuracy, knowledge-source validity, security/PII exposure, completeness, and whether specialist escalation is required — before choosing an action (Send AI Response, Edit & Send, Manual Response, Request Info, Escalate, Resolve, or Close).
- A full ticket status lifecycle — AI_RESOLUTION_READY → PENDING_AGENT_REVIEW → WAITING_FOR_CUSTOMER → RESOLVED → PENDING_CONFIRMATION → CLOSED on the happy path, with parallel AWAITING_CUSTOMER_INFO, ESCALATED, and REOPENED states for exceptions — so a ticket can never silently disappear.
- Customer confirmation and reopen handling: once a response is sent, the system waits for the customer to confirm the issue is solved or to report it is not; unconfirmed tickets follow a configured inactivity/closure policy, and confirmed-but-unresolved tickets return to an active or escalated state rather than closing prematurely.

- An escalation pathway to Team Lead, specialist, or another department for complex or low-confidence cases, with a defined owner at every stage and SLA monitoring (warning on approaching deadlines, a breach event and automatic escalation on SLA breach).
- A closure and feedback loop: final resolution notes are stored, the ticket is closed with complete status/audit history retained, the customer can rate the support experience, and poor AI outcomes or outdated knowledge sources are flagged back into the Knowledge Base for improvement — directly feeding the M2 knowledge-gap process and the M1 override-based retraining loop.
- New auditable data models — AITicketAnalysis, AIResolutionSuggestion, AgentReview, TicketResponse, Escalation, Resolution, CustomerFeedback, TicketStatusHistory, SLAEvent, and Notification — giving a single, queryable trail from AI suggestion through to customer feedback.
- Three dedicated portals: an expanded Customer Portal (ticket tracking, AI-support-response view, provide-information, resolution confirmation, feedback), a Support Agent Portal (AI Review Queue, AI Insights, Knowledge Suggestions, Escalations, SLA alerts), and an Admin Portal (AI Configuration/confidence thresholds, Routing Rules, SLA Management, Escalation Rules, Analytics, Audit Logs) plus a Manager Console for ticket assignment and Jira sync oversight.

Success definition for this milestone: a ticket is only considered successfully completed when an appropriate resolution is delivered, any required human validation has taken place, the customer's issue is resolved according to workflow policy, and the complete history — AI suggestion, agent decision, communications, escalations, and feedback — is retained and auditable.

3.6 Technology Stack

Layer	Technology
Frontend	React (TypeScript), Vite, Tailwind CSS, TanStack Query, Zod validation
Core Backend API	Django + Django REST Framework, JWT auth, Celery for async tasks
AI / ML Service	FastAPI, pydantic-settings, LightGBM / logistic regression classifier, cross-encoder reranker
Knowledge Store & Retrieval	MongoDB Atlas (Vector Search + Atlas Search / BM25 hybrid retrieval)
Generation Layer	LLM structured-output prompts (versioned YAML), Pydantic schema validation with repair-and-retry
Integrations	Jira REST API, transactional email provider
Workflow & Closure Layer	Confidence/policy engine, agent review queue, SLA timers, escalation and closure services (M4)
Infrastructure	Docker Compose (Postgres, Redis, MinIO, Mailhog) for local development

3.7 Real-World Impact

By separating deterministic logic (priority mapping, SLA computation, severity floors, confidence-threshold routing) from probabilistic components (classification, retrieval, generation), the platform produces an auditable, explainable pipeline suitable for enterprise IT support — every decision (why this severity, why this priority, why

this citation, why this ticket was auto-responded to versus escalated) can be traced and justified. The design explicitly favours a system that says 'I don't know, escalating to a human' over one that produces a confident but ungrounded answer, and Milestone 4 closes the loop by guaranteeing that no ticket is considered resolved without either an appropriate confidence-based automated pathway or a human agent's explicit validation and the customer's own confirmation. In a production setting, this reduces average resolution time on high-volume, well-documented ticket categories while preserving human review and accountability for anything ambiguous, high-risk, high-severity, or unconfirmed by the customer.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	M1: Repo/Docker setup, taxonomy-freezing session, category & priority model design, project scaffolding (frontend + backend).	Taxonomy and severity/priority matrix frozen; Docker Compose, Django and Vite/TS
Week	Activities Planned	Activities Completed
		skeletons stood up; historical ticket export requested.
Week 2	M1: Classification pipeline (preprocessing, feature extraction, fast/LLM dispatch), severity & priority logic, SLA engine, agent queue UI, evaluation report.	Ticket form, preprocessing (PII masking, email-artifact stripping), fast + LLM classification dispatch, severity overrides, priority matrix, SLA computation, and manual-override capture all delivered and evaluated against the hold-out set.
Week 3	M2: Knowledge base coverage audit, ingestion pipeline (loaders, chunking), Atlas Vector/Text index setup.	Coverage audit completed with the support lead; document loaders and heading-aware chunking built and unit-tested; Atlas vector and keyword indexes configured.
Week 4	M2: Hybrid retrieval (vector + keyword + RRF), query construction, reranking, grounded generation with citation enforcement, resolution panel UI.	Query-construction step, hybrid retrieval with RRF fusion, cross-encoder reranking, citation-enforced generation, and refusal path delivered; Recall@5 and citation-coverage evaluation completed.
Week 5	M3: Agent interfaces and orchestrator design, Diagnosis Agent, Knowledge Retrieval Agent wired to M2 RAG, Resolution Generation Agent, confidence validation and logging.	Multi-agent orchestrator, Diagnosis Agent, Knowledge Retrieval Agent (reusing M2 RAG), Resolution Agent, and activity logging implemented and unit-tested.
Week 6	M3: Jira authentication and sync, email provider and templates, ticket-created/resolution/escalation/resolved notifications, end-to-end integration testing.	Jira create/update/status-sync flow, transactional email templates and logging, and full end-to-end workflow test (ticket → diagnosis → retrieval → resolution → Jira → email) completed and validated.

Week 7	M4: Confidence/risk/policy engine, AI Review Queue, agent validation checklist and action buttons, ticket status lifecycle, escalation and SLA services.	Confidence-based routing (auto / agent-review / escalate) implemented with Admin-configurable thresholds; agent validation checklist, review queue, and action workflow (Send / Edit & Send / Manual / Request Info / Escalate / Resolve / Close) delivered; full status lifecycle and SLA warning/breach events wired in.
Week 8	M4: Customer confirmation and reopen handling, closure policy, feedback and knowledge-improvement loop, cross-milestone end-to-end testing, final demo and report.	Customer confirmation/reopen flow, closure and inactivity policy, feedback capture, and knowledge-gap flagging completed; full M1–M4 end-to-end demo rehearsed and delivered; final evaluation reports assembled; internship completion report prepared.

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Taxonomy, severity, and priority model frozen; project scaffolding for frontend, Django backend, and FastAPI AI service established.	End of Week 1
Milestone	Description	Date Achieved
Milestone 1 Complete	Ticket intake, PII-safe preprocessing, fast/LLM classification dispatch, severity overrides, deterministic priority mapping, and SLA computation delivered and evaluated ($\geq 90\%$ classification accuracy, $\geq 85\%$ severity accuracy target).	End of Week 2
Mid-Term Review / Milestone 2 Complete	Knowledge base ingestion, hybrid (vector + keyword) retrieval with RRF fusion and reranking, and citation-enforced grounded generation delivered and evaluated ($\text{Recall}@5 \geq 0.85$, 100% cited resolutions).	End of Week 4
Milestone 3 Complete	Multi-agent orchestration (Diagnosis, Retrieval, Resolution, Escalation), Jira integration, and email automation delivered and validated end to end.	End of Week 6
Milestone 4 Complete	Confidence-based auto/agent-review/escalation routing, agent validation workflow, ticket status lifecycle, customer confirmation, controlled closure, and feedback loop delivered and validated.	End of Week 8
Final Submission	Consolidated evaluation evidence, Definition-of-Done checklists, and full system documentation compiled across all four milestones.	Week 8

Presentation	End-to-end live demonstration of the complete SupportPilot pipeline: ticket submission → classification → grounded resolution → agent workflow → Jira/email → agent validation → customer confirmation → closure.	Week 8
--------------	---	--------

5b. Project execution details

The project was executed in four tightly scoped, sequential milestones, each with its own frozen requirements document, day-by-day (or week-by-week) build plan, and signed-off Definition of Done before the next milestone began. This staged approach prevented scope creep — for example, the temptation to begin building the RAG/retrieval system during Milestone 1, or to build the full multi-agent orchestration graph during Milestone 2, was explicitly identified and avoided in the specifications, keeping each milestone's deliverable testable in isolation. Milestone 4 continued this discipline by explicitly not re-doing classification, retrieval, or agent orchestration — it consumed the outputs of AITicketAnalysis and AIResolutionSuggestion produced upstream and focused purely on validation, communication, escalation, and closure.

Execution principles followed throughout

- Contract-first development: shared schemas (e.g., the Zod ticket schema, the classification/severity/SLA method signatures, and the M4 AIResolutionSuggestion / AgentReview / Resolution data models) were agreed between frontend and backend before implementation began, so both sides could build in parallel against a stable interface.
- Fast-path/slow-path routing: cheap, deterministic, or lightweight ML methods handled the bulk of routine cases (ticket classification, retrieval reranking cut-offs, high-confidence auto-response eligibility), while expensive LLM calls and full human review were reserved for genuinely uncertain or high-stakes cases — controlling both latency and cost at scale.
- Evaluation before demonstration: each milestone's exit criteria were measured against a frozen, time-split hold-out or golden set before being presented, with confusion matrices, recall/precision breakdowns, citation-coverage numbers, and (for M4) confidence-routing accuracy and escalation-rate reporting rather than a single headline metric.
- Human-in-the-loop by design: manual classification overrides (M1), a mandatory 'draft — not sent' state on all generated resolutions (M2), an escalation path for low-confidence or unresolved cases (M3), and a full agent-validation checklist with configurable confidence thresholds and mandatory review for sensitive cases (M4) ensured the system augmented rather than replaced human judgement at every stage.
- Auditability: every classification, retrieval decision, generated resolution, agent execution, Jira action, email send, agent review decision, escalation, and closure was logged (Activity_Logs, retrieval_logs, Agent_Executions, Email_Logs, TicketStatusHistory, SLAEvent), so any output could be traced back to its inputs and reasoning.

Risk management

Several cross-milestone risks were actively managed. Data/content risk: Milestone 1's exit criteria depended on $\geq 2,500$ labelled historical tickets, and Milestone 2's depended on adequate knowledge base coverage for the highest-volume ticket categories — both were escalated early (day 1/day 3) with a documented fallback (an LLM-only classification path, and prioritised content-gap writing) rather than being discovered late.

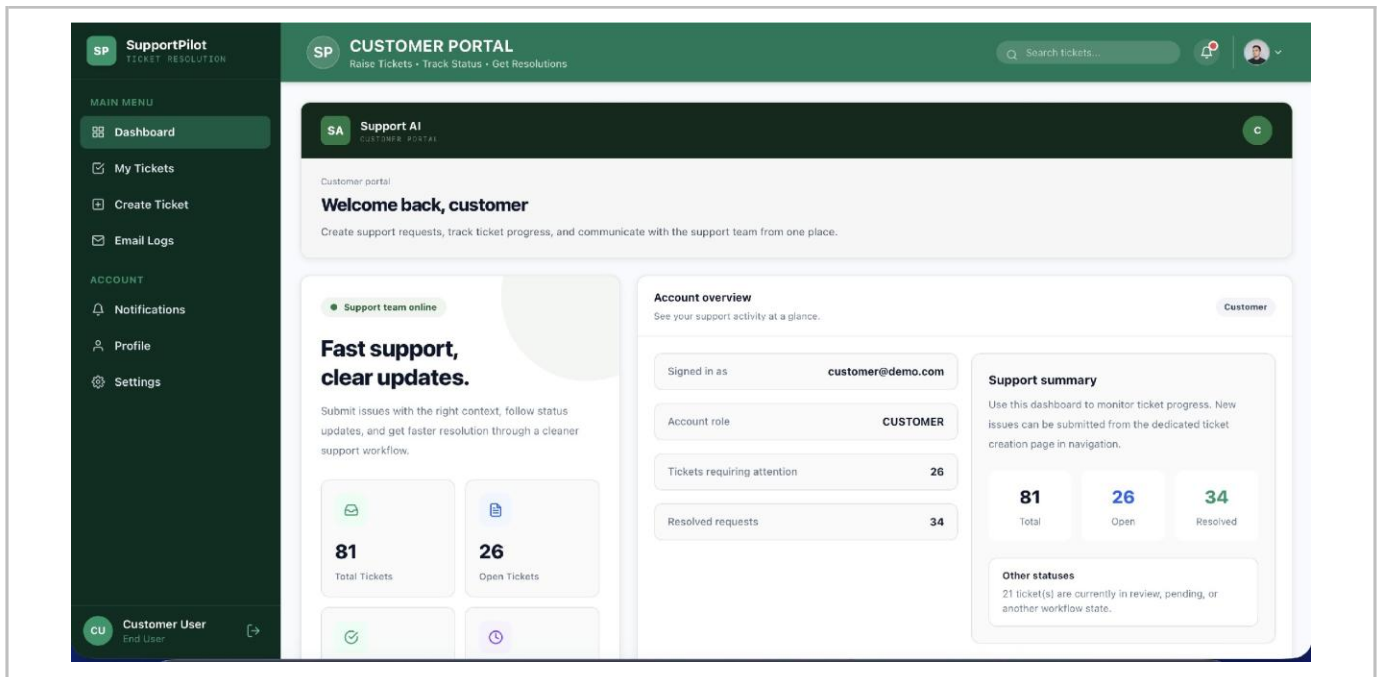
Taxonomy/data freezing risk: the category taxonomy (Milestone 1) and the retrieval hold-out set (Milestones 1 and 2) were explicitly frozen on fixed days, since any late change would silently invalidate previously collected

evaluation evidence. Closure/accountability risk: Milestone 4 explicitly guarded against tickets 'silently disappearing' by mandating a defined closure policy, a reopen path for unresolved confirmations, and full status-history retention, and against unsafe automation by making confidence thresholds Admin-configurable rather than fixed, and by requiring human review for sensitive cases regardless of confidence score.

6. Snapshots / Screenshots

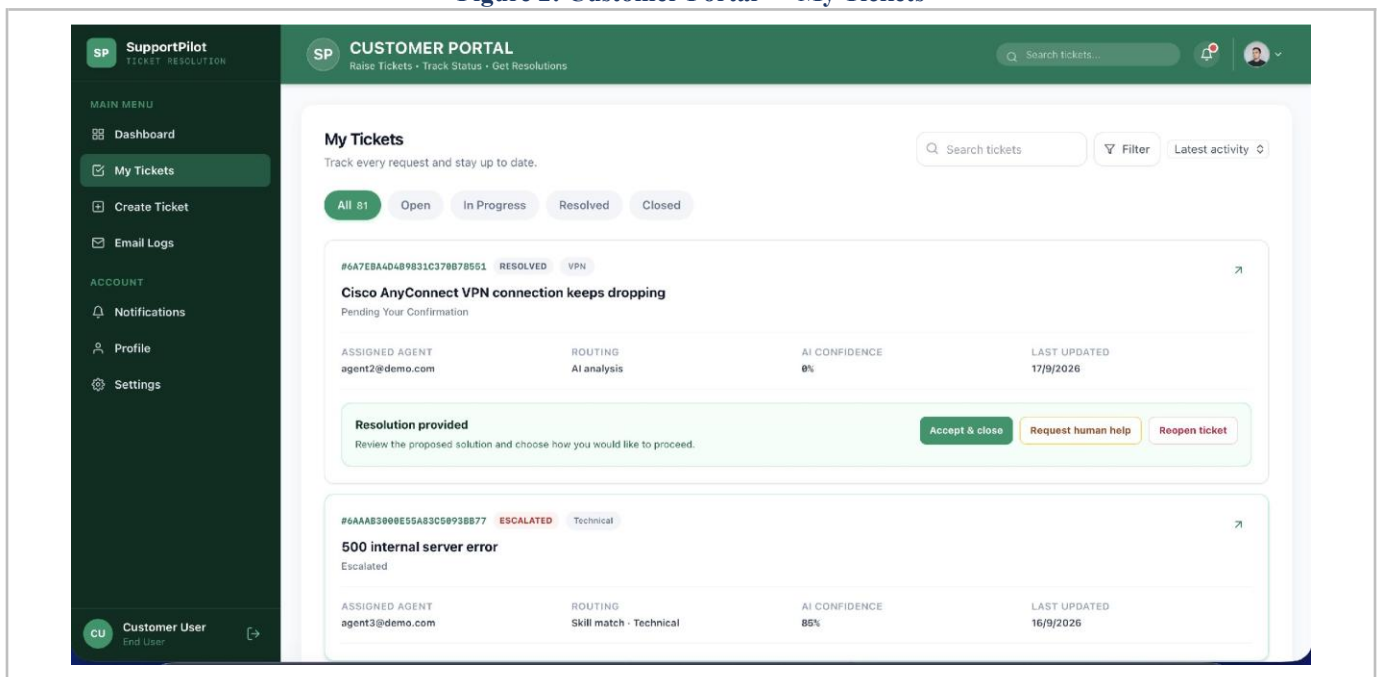
The following screenshots show the working SupportPilot application across its Customer, Support Agent, Manager, and Admin portals, demonstrating the end-to-end flow from ticket creation through AI classification, agent review, escalation, and closure.

Figure 1: Customer Portal — Create New Ticket



The customer-facing intake form: subject, description, category, and impact questions ('Who is affected?') that feed the M1 classification and severity/priority engine.

Figure 2: Customer Portal — My Tickets



Ticket list showing AI confidence, routing, and status per ticket, along with the M4 customer-confirmation actions (Accept & Close, Request Human Help, Reopen Ticket) on a resolved ticket.

Figure 3: Customer Portal — Dashboard

Account overview with total, open, and resolved ticket counts, giving the customer a quick snapshot of their support activity.

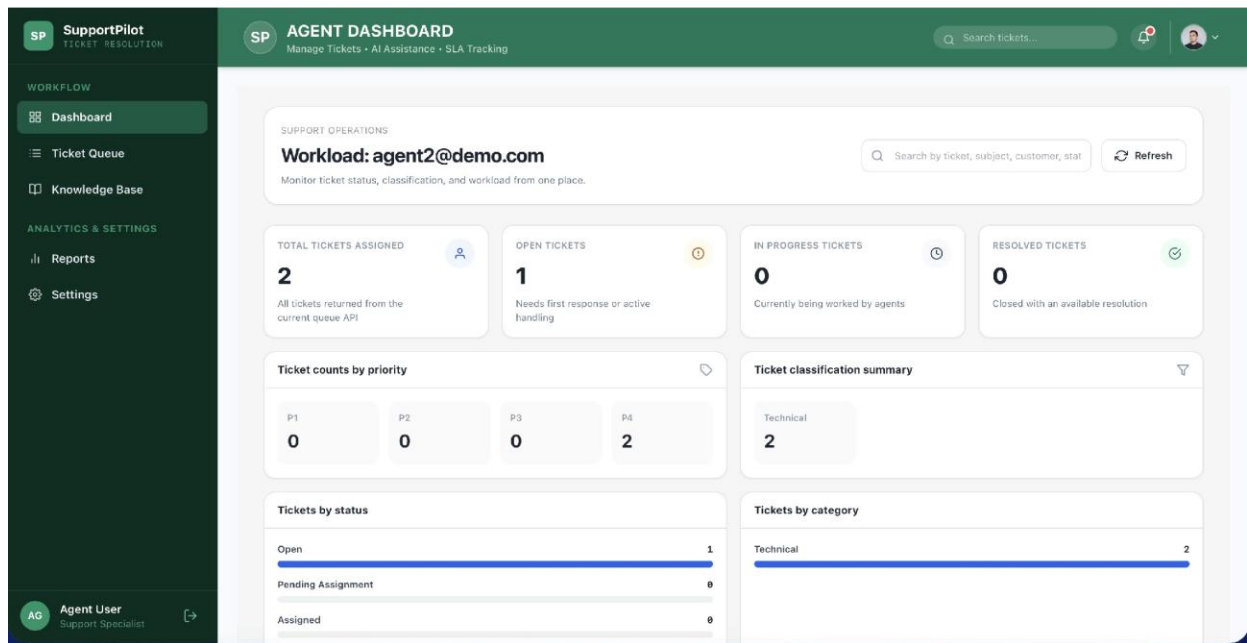
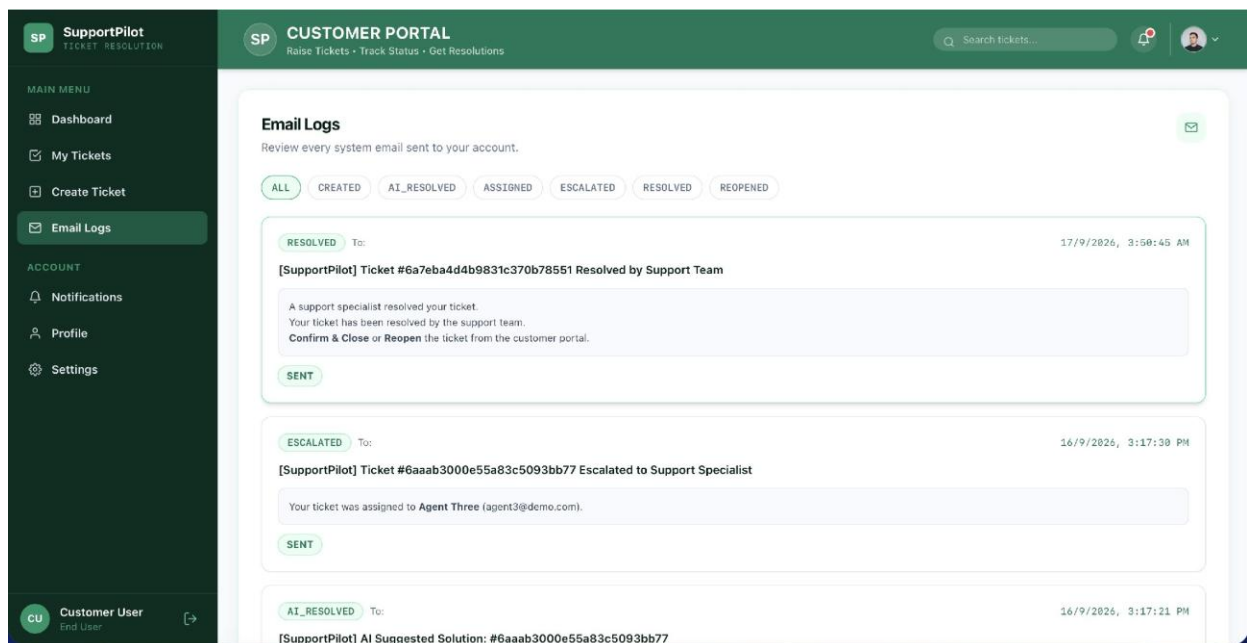


Figure 4: Customer Portal — Email Logs



Automated notification history (Resolved, Escalated, AI Resolved) confirming that every customer communication is sent via the email automation layer and logged for audit, as specified in M3/M4.

Figure 5: Support Agent Portal — Dashboard

Agent workload view: open/in-progress/resolved ticket counts, tickets by priority (P1–P4), and classification/category summary for the signed-in agent.

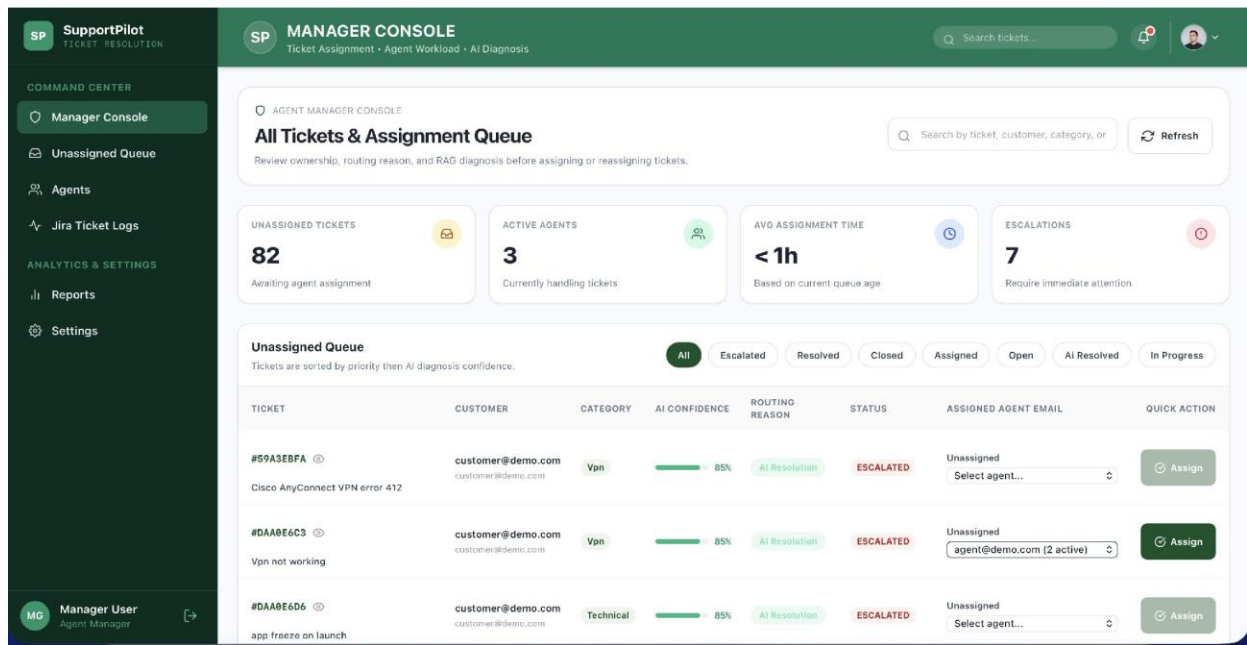
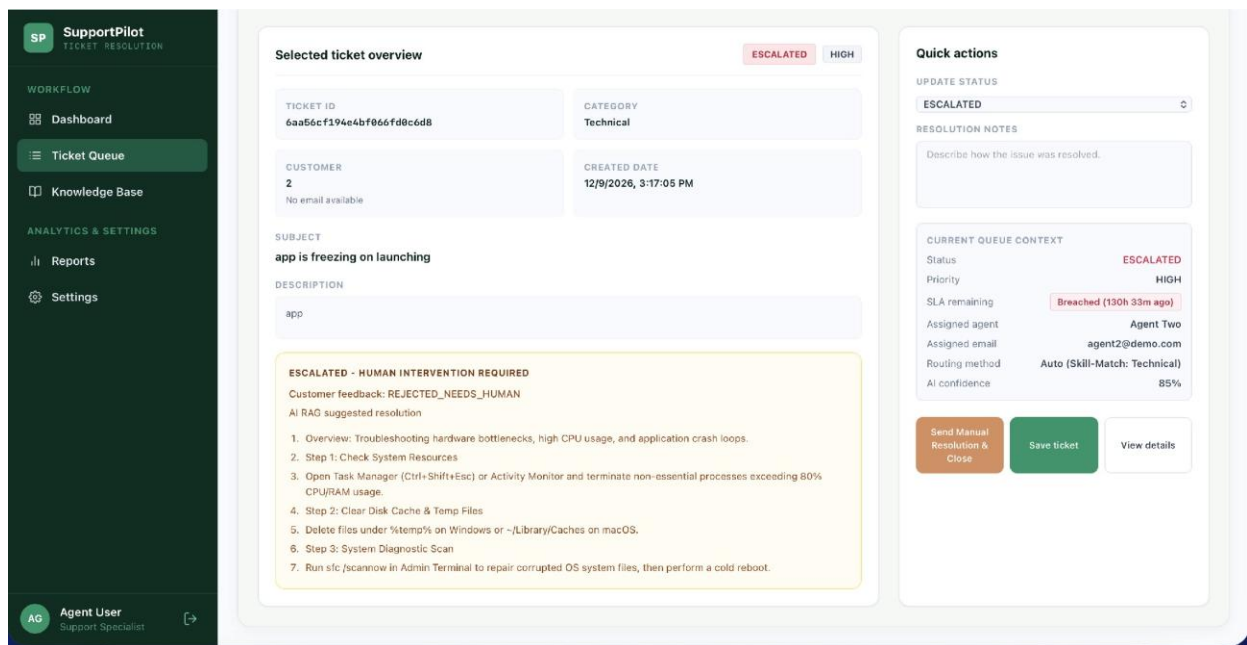


Figure 6: Support Agent Portal — Ticket Queue Detail



An escalated, SLA-breached ticket showing the AI (RAG) suggested resolution, current queue context (routing method, AI confidence), and agent quick actions (Send Manual Resolution & Close, Save Ticket) — the M4 agent-validation workflow in action.

Figure 7: Manager Console — All Tickets & Assignment Queue

Manager-level view of unassigned tickets, active agents, average assignment time, and escalations, with AI confidence and routing reason shown per ticket before assignment.

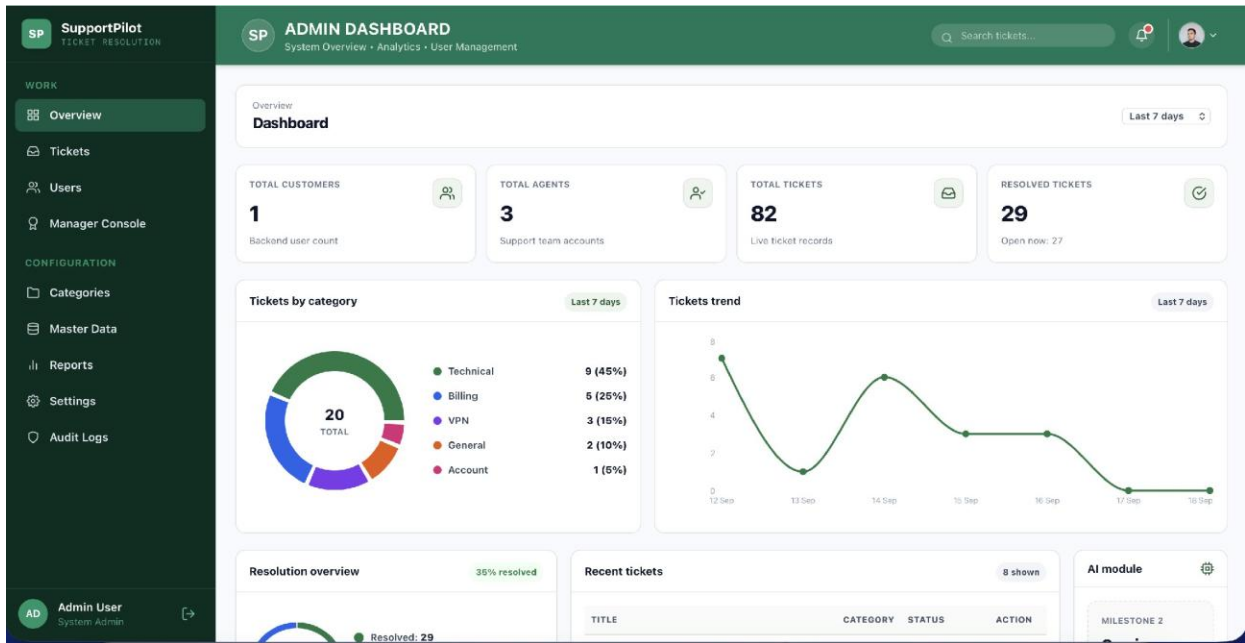
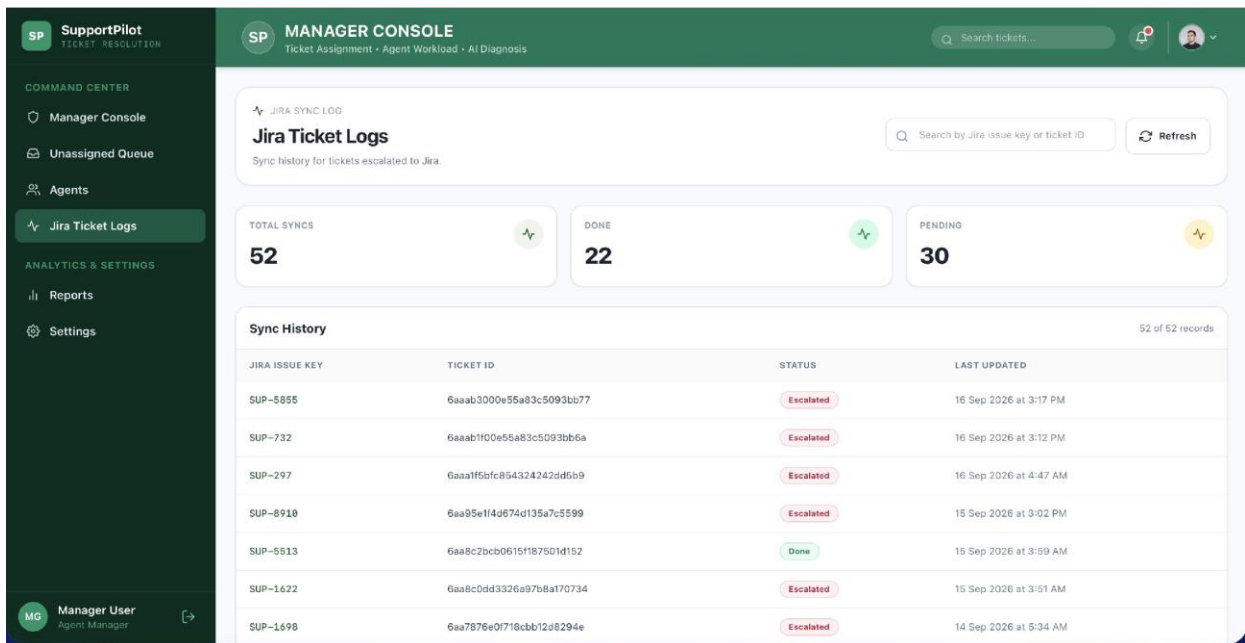


Figure 8: Manager Console — Jira Ticket Logs

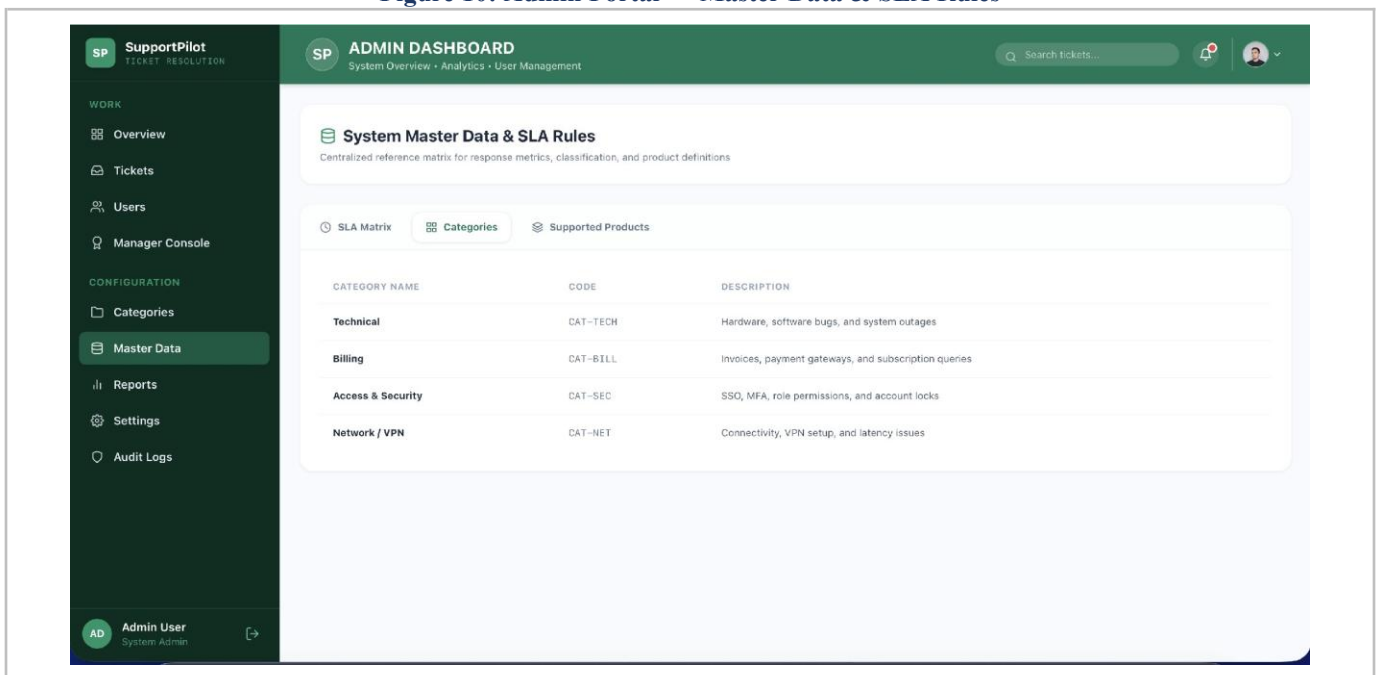


Sync history between SupportPilot and Jira for escalated tickets, showing total syncs, completed vs. pending status, and per-issue sync timestamps — the M3 Jira integration audit trail.

Figure 9: Admin Portal — Dashboard

System-wide analytics: total customers/agents/tickets, resolution rate, tickets-by-category breakdown, and a 7-day ticket trend line.

Figure 10: Admin Portal — Master Data & SLA Rules



Centralised configuration of ticket categories (Technical, Billing, Access & Security, Network/VPN) that drive the M1 taxonomy, routing, and SLA matrix.

7. Challenges Faced

- Class imbalance and confusability in ticket classification: categories like NETWORK and VPN were frequently confused since VPN is technically a subset of network issues. This was mitigated by keeping VPN as an explicit, separate top-level category (since it is high-volume and highly template-able for

auto-resolution) and by adding explicit keyword features and class-weighting rather than oversampling, which risked overfitting on near-duplicate ticket text.

- Retrieval quality being capped by knowledge base content, not model quality: early testing showed that raw ticket text produced poor retrieval (Recall@5 around 0.55–0.65) purely because ticket descriptions are noisy. This was resolved by introducing a dedicated query-construction step that distills a messy ticket into 2–3 focused search queries before retrieval, which alone improved Recall@5 by roughly 0.15–0.20.
- Preventing hallucinated or ungrounded resolutions: initial generation attempts occasionally produced plausible-sounding steps not actually supported by retrieved content. This was addressed with independent citation-resolution logic (dropping any step whose cited source could not be resolved against chunks actually sent to the model) and a deliberate refusal path when context was insufficient, rather than relying solely on prompt instructions.
- Avoiding scope creep across milestones: there was recurring temptation to start Milestone 2 (RAG) work during Milestone 1, and to build the full multi-agent orchestration graph during Milestone 2. Strict adherence to each milestone's frozen scope document and Definition of Done checklist kept the phases decoupled and testable.
- Eventual consistency in MongoDB Atlas Search/Vector Search: newly ingested chunks were not immediately queryable after insertion, which initially caused ingestion-verification and integration tests to report false failures. This was resolved by adding a polling/retry loop to the verification and test harnesses instead of a single query-and-fail check.
- Avoiding user-set urgency bias: an early design considered letting requesters directly select ticket urgency, but this was recognised as a bias risk (nearly every self-reported urgency trends toward 'Critical'). The fix was to capture self-reported urgency only as a low-weight model feature and compute the actual severity/priority deterministically.
- Choosing safe, defensible confidence thresholds for M4 auto-response: a fixed universal cut-off risked either over-automating risky answers or under-automating safe, high-volume ones. This was resolved by making thresholds Admin-configurable per category/policy, and by hard-requiring human review for sensitive cases (e.g., security, billing, PII exposure) irrespective of the confidence score returned.
- Preventing tickets from silently going stale after a response was sent: without a defined policy, tickets awaiting customer confirmation could linger indefinitely. This was solved with an explicit inactivity/closure policy and a PENDING_CONFIRMATION status, so every ticket has a deterministic path to CLOSED or REOPENED rather than an open-ended wait.

8. Learnings & Skills Acquired

Technical skills

- End-to-end applied ML: feature engineering for text classification, handling class imbalance with class weights, confidence calibration (Platt scaling / isotonic regression), and time-based (not random) train/hold-out splitting to prevent data leakage.
- Retrieval-Augmented Generation architecture: hybrid (vector + keyword) search, Reciprocal Rank Fusion, cross-encoder reranking, heading-aware document chunking, and citation-enforced structured generation with schema validation and repair-and-retry logic.

- Multi-agent system design: designing an orchestrator with clear agent responsibilities (Diagnosis, Retrieval, Resolution, Escalation), workflow state tracking, and confidence-gated decision points.
- Human-in-the-loop workflow engineering: designing confidence/risk/policy decision trees, agent review queues and validation checklists, ticket status-lifecycle state machines, SLA timer/breach handling, and closure/reopen policies that guarantee no ticket is lost or left in an ambiguous state.
- Full-stack engineering: React/TypeScript frontend architecture with a strict pages/features/lib separation enforced by lint rules, Django REST Framework backend design (RBAC, state machines, cursor pagination), and FastAPI microservice design for AI workloads.
- Third-party integration engineering: Jira REST API integration with bidirectional status synchronisation, and transactional email automation with delivery logging.
- MongoDB Atlas Vector Search and Atlas Search configuration, including index design and handling of eventual consistency in test and verification harnesses.

Domain knowledge

- IT service management concepts: severity/priority/SLA frameworks, business-hours vs. 24×7 SLA calendars, and ITSM ticket state machines, extended through M4 into confirmation, reopen, and closure governance.
- Practical RAG evaluation methodology: Recall@k, MRR, citation coverage, and the use of a dedicated 'refusal set' to test whether a system correctly declines to answer rather than always producing confident output.
- Support-operations quality management: role-based access design (customer, agent, team lead, manager, admin), confidence-threshold governance, escalation ownership rules, and using customer feedback and knowledge-gap flags to drive continuous improvement of both the AI and the knowledge base.

Soft skills and ways of working

- Scope discipline: writing and following a frozen scope boundary per milestone, and explicitly documenting what was deliberately deferred rather than silently dropped (carry-forward registers).
- Evidence-based reporting: presenting exit-criteria evidence (confusion matrices, recall/precision, evaluation reports, resolution/escalation rates) rather than only a single summary metric, and being transparent about shortfalls (e.g., a fallback plan when labelled data volume was at risk).
- Cross-functional collaboration: working with a (simulated) support desk lead and knowledge manager to freeze a taxonomy, audit knowledge base coverage, build evaluation golden sets, and define agent validation checklists and escalation ownership — skills directly transferable to real product/engineering collaboration with subject-matter experts.

9. Testimonials from team

This internship project provided a rare opportunity to work across the full lifecycle of a modern AI product — from deterministic backend logic and classical ML, through retrieval-augmented generation and multi-agent orchestration, to the human-in-the-loop validation and closure workflow that actually makes such a system trustworthy in production — within a single, coherent 8-week build. The milestone-based structure, with clearly frozen scope and measurable exit criteria at each stage, closely mirrored how real production AI systems are shipped incrementally and responsibly, with human oversight retained at every stage where model confidence

could not be guaranteed. Milestone 4 in particular was a valuable reminder that an AI system is only as useful as its ability to know when to hand off to a human, communicate clearly with the customer, and leave behind a clean, auditable record of what actually happened.

10. Conclusion

Over the course of this internship, SupportPilot was designed and built as a complete, four-milestone AI-powered IT support platform: Milestone 1 delivered accurate, auditable ticket classification and SLA management; Milestone 2 layered in grounded, cited knowledge retrieval and resolution generation on top of that classification; Milestone 3 unified both into a coordinated multi-agent workflow connected to real enterprise tools (Jira and email); and Milestone 4 closed the loop by converting AI-generated recommendations into verified, human-approved, and formally closed customer resolutions, with escalation, confirmation, and feedback built in as first-class outcomes rather than afterthoughts. The screenshots in Section 6 confirm that this design was carried through to a working, end-to-end application across the Customer, Support Agent, Manager, and Admin portals.

The experience reinforced that building trustworthy AI systems is as much about scope discipline, evaluation rigor, and human-in-the-loop design as it is about model selection or prompt engineering — and that a project is not truly complete until there is a measurable, auditable definition of a 'closed' and 'successful' outcome, not just a generated answer. These skills — full-stack AI product development, RAG system design, multi-agent orchestration, and human-in-the-loop workflow and closure design — directly extend prior academic learning in software engineering and machine learning into a production-oriented context, and align closely with a career direction in applied AI/ML engineering and AI product development.

11. Acknowledgements

I would like to sincerely thank the organisation for the opportunity to work on this project, and my mentor(s) for their guidance, structured feedback, and the well-scoped milestone specifications that made a genuinely complex, four-stage AI system achievable within the internship timeline. I am also grateful to the support desk and knowledge management stakeholders (simulated within the project) whose input shaped the category taxonomy, severity model, knowledge base coverage decisions, and agent validation checklist, and to my fellow team members in Team 1 for their collaboration throughout the internship.